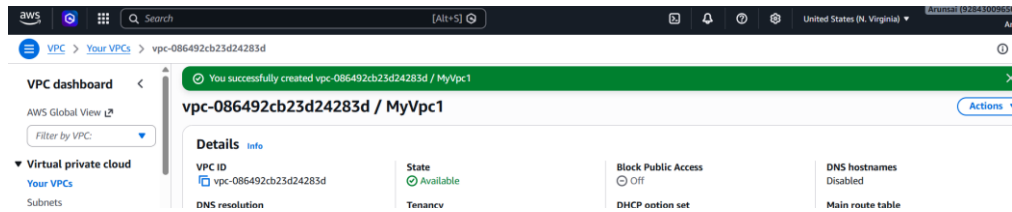


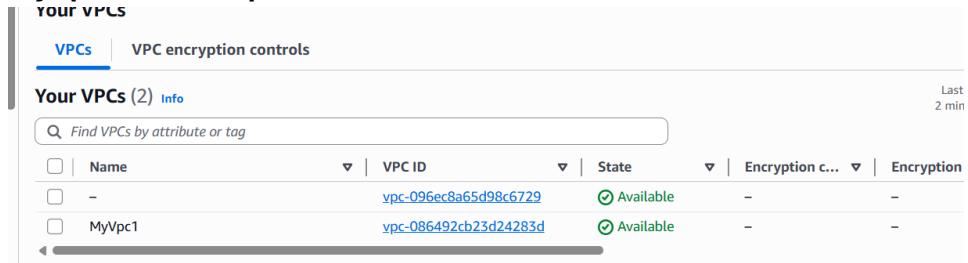
AWS AUTOSCALING

1. Create one VPC in N. Virginia region.

- Created a New VPC in N. Virginia Region



- MyVpc1 is new Vpc created



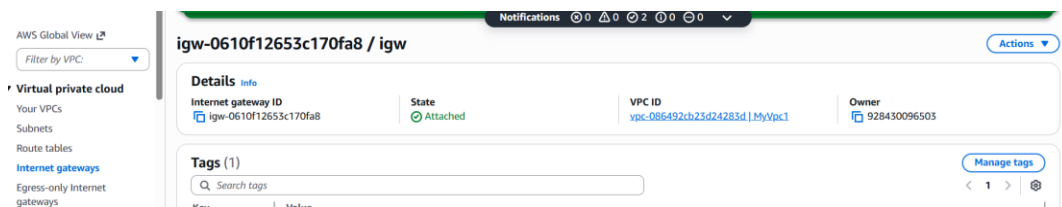
2. Create two subnets: one public subnet and one private subnet.

- Created 1 public and 1 private subnets

☐	-	subnet-0a25ef5df35935262	Available	vpc-096ec8a65d98c6729	
☐	Public1	subnet-0152e618d7bae7db1	Available	vpc-086492cb23d24283d My...	
☐	Private1	subnet-05086d992c829af32	Available	vpc-086492cb23d24283d My...	

3. Attach an IGW to the VPC.

- Created a new IGW and attach to myvpc1



4. Create one public route table (RT) and one private route table.

- Created and 1 Public and 1 Private Route table

Details

Route tables (2) [Info](#)

Last updated less than a minute ago [Action](#)

Find route tables by attribute or tag

Name = Public-Route X Name = Private-Route X Clear filters

☐	Name	Route table ID	Explicit subnet associ...	Edge associations	Main
☐	Public-Route	rtb-06c73f5f194286469	2 subnets	–	No
☐	Private-Route	rtb-066a9dfb8c0e8d9ae	2 subnets	–	No

5. Deploy a NAT gateway in the public subnet and attach the NAT gateway to the private subnet.

- Create a NAT gateway in public subnet

[VPC](#) > [NAT gateways](#) > nat-14ef1dcfc99443862

VPC dashboard < [AWS Global View](#)

Filter by VPC: [▼](#)

Virtual private cloud

- Your VPCs
- Subnets
- Route tables
- Internet gateways
- Egress-only Internet gateways

NAT gateway nat-14ef1dcfc99443862 | NAT-1 was created successfully.

nat-14ef1dcfc99443862 / NAT-1

Details

NAT gateway ID nat-14ef1dcfc99443862	Availability mode Regional	State ⌵ Pending
NAT gateway ARN arn:aws:ec2:us-east-1:928430096503:natgateway/nat-14ef1dcfc99443862	Connectivity type Public	Created ⌵ Monday, 27 April 2026 at 15:12:59 GMT+5:30
VPC vpc-086492cb23d24283d / MyVpc1	Method of EIP allocation Automatic	

- Attach NAT gateway to the private subnet

[Routes](#) | [Subnet associations](#) | [Edge associations](#) | [Route propagation](#) | [Tags](#)

Routes (2) [Both](#) [Edit routes](#)

Filter routes

Destination	Target	Status	Propagated	Route Origin
0.0.0.0/0	nat-14ef1dcfc99443862	Active	No	Create Route
192.168.0.0/18	local	Active	No	Create Route Table

6. Create two instances, one in the public subnet and one in the private subnet.

- Created two Instances 1 is public and 1 is private subnet

[Running](#) [Find Instance by attribute or tag \(case-sensitive\)](#)

Instance state = running X Clear filters

☐	Name	Instance ID	Instance state	Instance type
☐	Public-EC2	i-04c738b3c0430274c	Running	t3.micro
☐	Private-EC2	i-0c12edac4eeaacd2a	Running	t3.micro

7. Deploy Apache server on both EC2 instances with a sample index.html file.

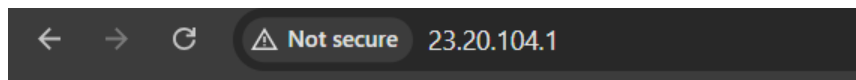
- Run the Public EC2 and install Apache server

```

pgout
Connection to 192.168.3.43 closed.
ec2-user@ip-192-168-1-173 ~]$ sudo systemctl status httpd
httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: active (running) since Mon 2026-04-27 10:12:44 UTC; 32min ago
     Docs: man:httpd.service(8)
   Main PID: 26995 (httpd)
    Status: "Total requests: 3; Idle/Busy workers 100/0; Requests/sec: 0.00155; Bytes"
      Tasks: 230 (limit: 1014)
    Memory: 16.9M
       CPU: 2.384s
    CGroup: /system.slice/httpd.service
            └─26995 /usr/sbin/httpd -DFOREGROUND
              └─26996 /usr/sbin/httpd -DFOREGROUND
                └─26997 /usr/sbin/httpd -DFOREGROUND

```

- **Check in browser public**



It works!

- **Created index.html page and save it**

```

ec2-user@ip-192-168-1-173:~$ nano index.html
GNU nano 8.3 index.html
<!DOCTYPE html>
<html>
<head>
  <title>My AWS Web Server</title>
</head>
<body>
  <h1>Public EC2 Web Server</h1>
  <p>This is running from Apache on AWS!</p>
</body>
</html>

```

- **Now again refresh the page check the new index file is created**



Public EC2 Web Server

This is running from Apache on AWS!

- **Now private EC2 created**
- Run the public EC2 and created a vi file and paste the private pem key in that

- And given the permissions and with ssh -i pem key and user id and private id
- **Connect the private EC2 in public EC2 with pem key**

```
[ec2-user@ip-192.168.3.43: ~]$ ssh -i aws.pem ec2-user@192.168.3.43
```

```
Permission denied (publickey,gssapi-keyex,gssapi-with-mic).
```

```
[ec2-user@ip-192.168-1-173 ~]$ vi aws.pem
```

```
[ec2-user@ip-192.168-1-173 ~]$ ssh -i aws.pem ec2-user@192.168.3.43
```

```
#  
##### Amazon Linux 2023  
#####  
###|  
#/  
V~'--> https://aws.amazon.com/linux/amazon-linux-2023  
  
_.  
/_m/'-->
```

```
[ec2-user@ip-192-168-3-43 ~]$ sudo yum install httpd -y
```

```
Amazon Linux 2023 Kernel Livepatch repository      246 kB/s |   31 kB    00:00  
Last metadata expiration check: 0:00:01 ago on Mon Apr 27 10:36:28 2026.  
Dependencies resolved.
```

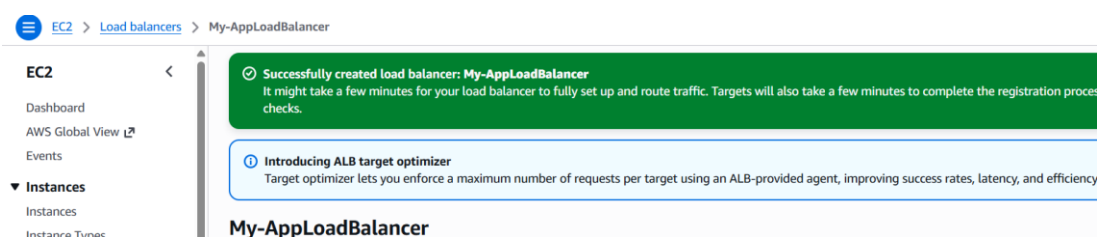
Package	Arch	Version	Repository	Size
Installing:				
httpd	x86_64	2.4.66-1.amzn2023.0.1	amazonlinux	47 k
Installing dependencies:				

- **Create a index file for private and save it**
- **Now check the private is running with curl url**

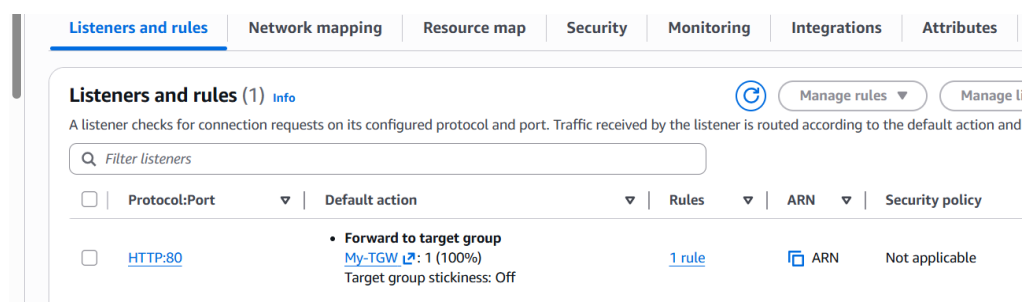
```
[ec2-user@ip-192-168-3-43 ~]$ sudo systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service.
systemd/system/httpd.service.
[ec2-user@ip-192-168-3-43 ~]$ cd /var/www/html
[ec2-user@ip-192-168-3-43 html]$ sudo nano index.html
[ec2-user@ip-192-168-3-43 html]$ sudo systemctl restart httpd
[ec2-user@ip-192-168-3-43 html]$ ls
index.html
[ec2-user@ip-192-168-3-43 html]$
```

8. Create one application load balancer and attach it to both EC2 instances.

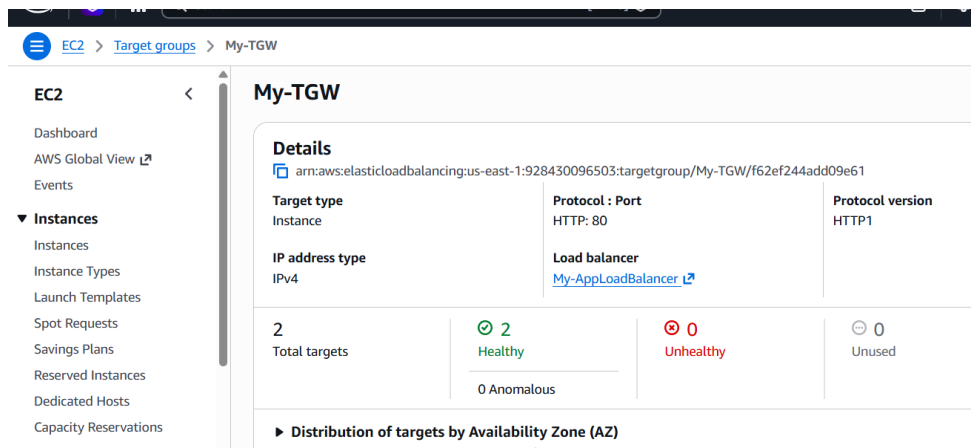
- **Created one Application load balancer**



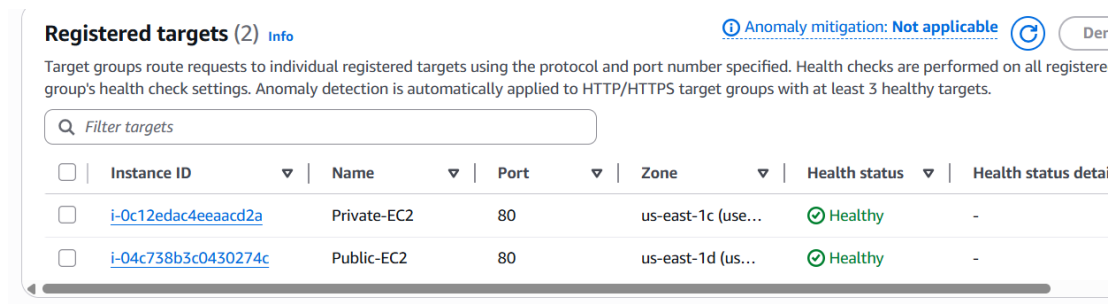
- **Given HTTP 80 and security group created and attached**



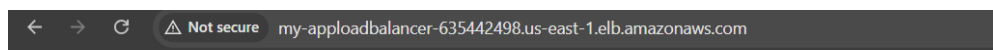
- Created target group and to it vpc attached and also public subnets



- We can see the registers subnets public and private



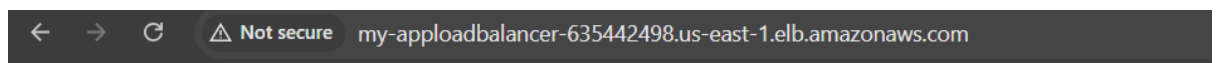
- Now with DNS url from Application Load Balancer we can check in browser public and private index.html



Public EC2 Web Server

This is running from Apache on AWS!

- Now we can check private index.html which we create in earlier

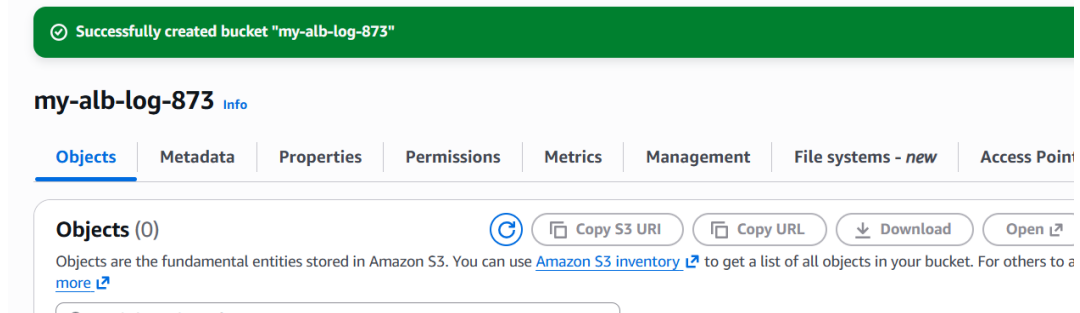


Private EC2 Server

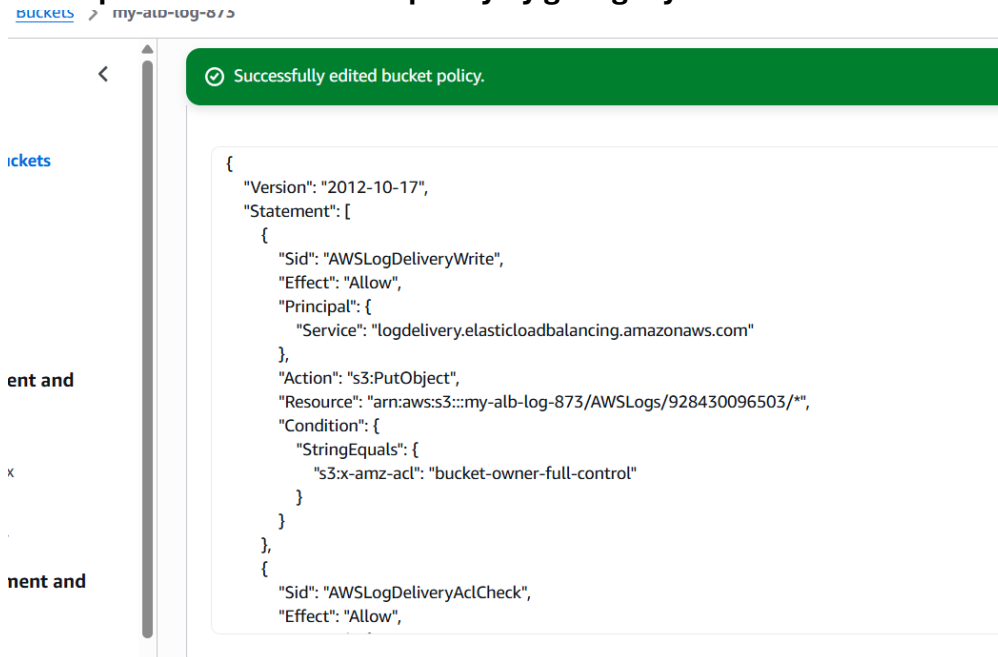
This is Private Subnet

9. Store application load balancer logs in S3.

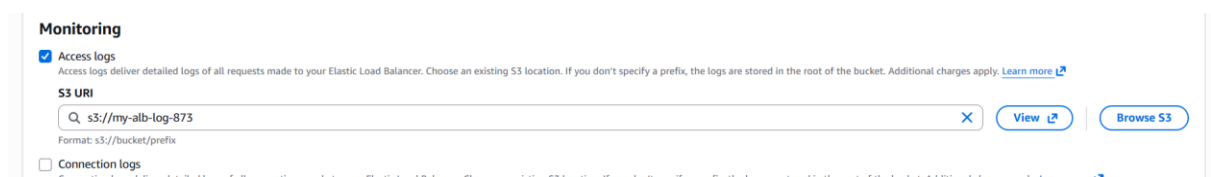
- Created new bucket in s3 with unique and same region of ALB



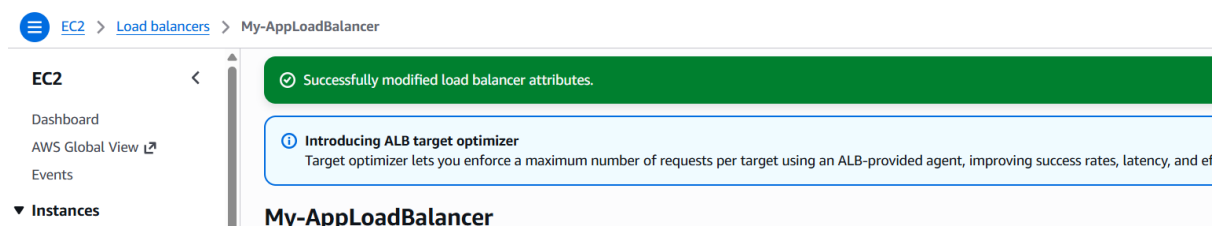
- Given permission in bucket policy by giving my account id and bucket name



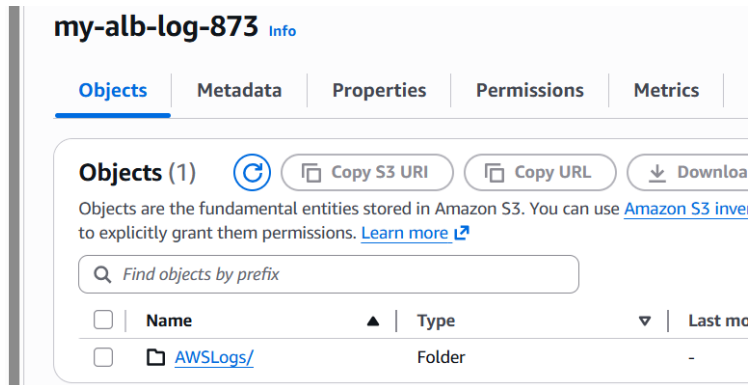
- Now in App Load Bal open in attributes on the logs and select my new bucket



- ALP Attributes logs are on

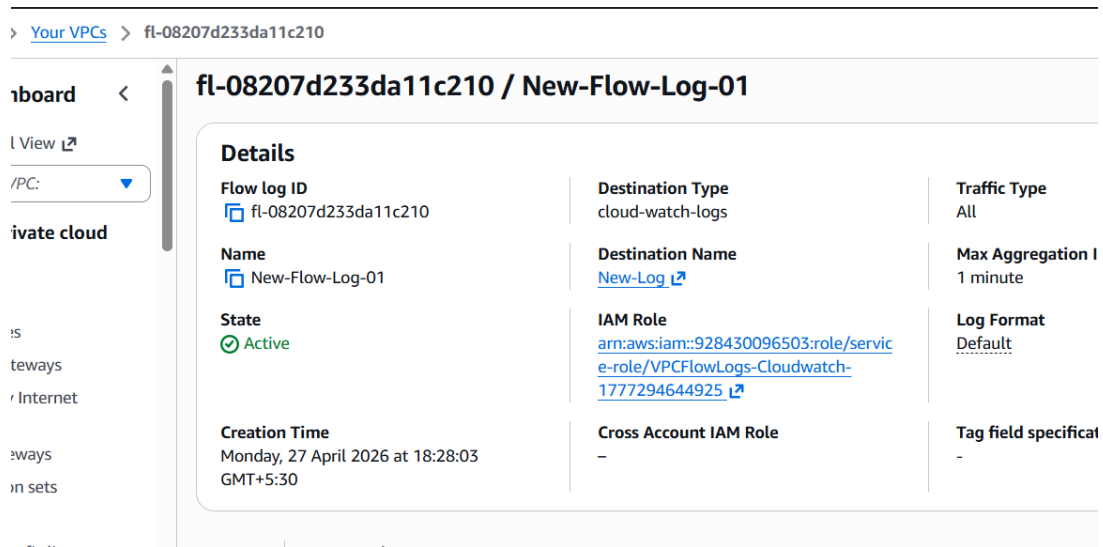


- Now we can check in bucket folder the logs created

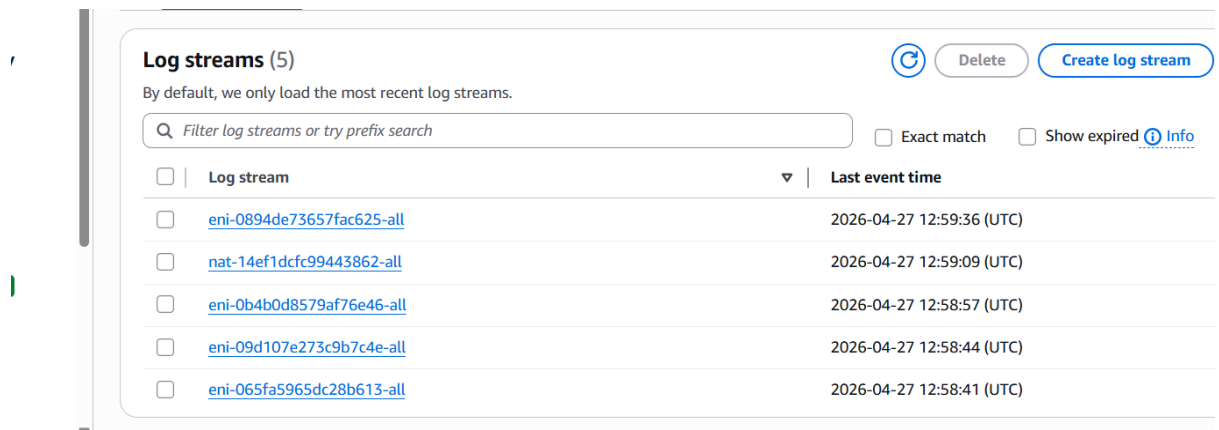


10. Store the VPC flow logs in a CloudWatch log group.

- Create a new flow log in Vpc and send to CloudWatch from vpc
- Create a new IAM Role attached CloudWatch Access and save



- In CloudWatch check the logs files for that



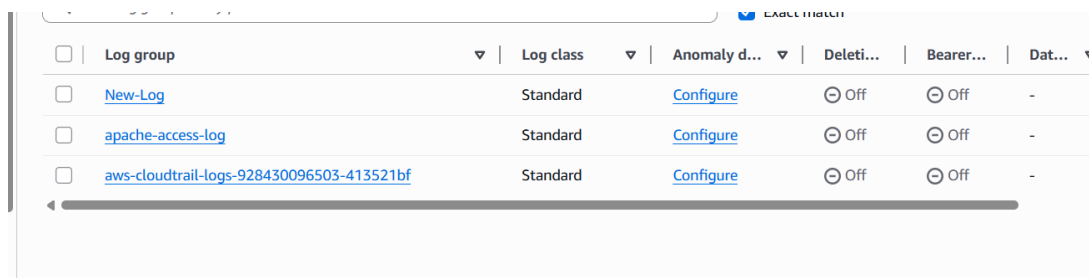
11. Create monitoring dashboards to monitor CPU utilization and to monitor the Apache service.

- Run the Public EC2 and install the cloud -watch agent

```
[ec2-user@ip-192-168-1-173 ~]$ sudo yum install amazon-cloudwatch-agent -y
Last metadata expiration check: 3:21:27 ago on Mon Apr 27 09:59:29 2026.
Dependencies resolved.
=====
Package                        Arch      Version                        Repository      Size
=====
Installing:
amazon-cloudwatch-agent        x86_64    1.300064.2-1.amzn2023        amazonlinux     68 M
=====
Transaction Summary
=====
Install 1 Package

Total download size: 68 M
Installed size: 228 M
Downloading Packages:
amazon-cloudwatch-agent-1.300064.2-1.amzn2023.x86_64 69 MB/s | 68 MB  00:00
-----
Total                                           65 MB/s | 68 MB  00:01
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing                : 1/1
  Running scriptlet: amazon-cloudwatch-agent-1.300064.2-1.amzn2023.x86_64 1/1
```

- We can get the apache log files in CloudWatch



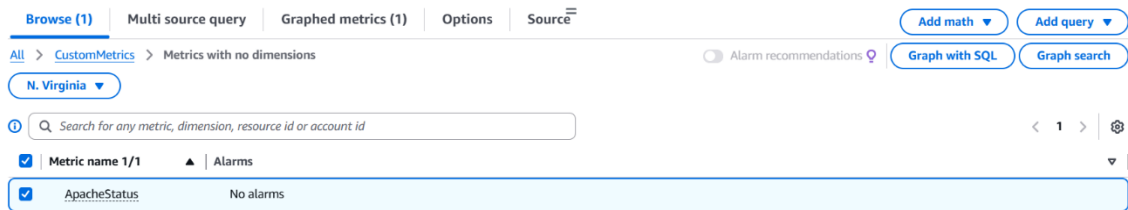
- Create a file of apache server and save if it runs 1 and fail 0
- Give access permissions and also other permissions to

```
ec2-user@ip-192-168-1-173 ~]$ sudo systemctl restart amazon-cloudwatch-agent
ec2-user@ip-192-168-1-173 ~]$ nano apache-check.sh
ec2-user@ip-192-168-1-173 ~]$ chmod +x apache-check.sh
ec2-user@ip-192-168-1-173 ~]$ ./apache-check.sh

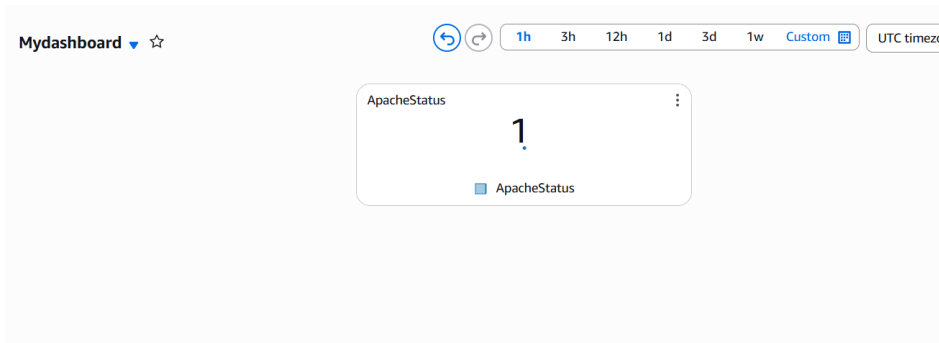
ec2-user@ip-192-168-1-173 ~]$ aws configure
WS Access Key ID [None]: AKIA5QKV7AB3SMJE02X6
WS Secret Access Key [None]: rStUFioWEXm1eS52npetizj4Baz8SL25K8X9AAAbw
default region name [None]: us-east-1
default output format [None]: json
ec2-user@ip-192-168-1-173 ~]$ nano apache-check.sh
ec2-user@ip-192-168-1-173 ~]$ ./apache-check.sh

ec2-user@ip-192-168-1-173 ~]$ ls
apache-check.sh  aws.pem
ec2-user@ip-192-168-1-173 ~]$ VALUE=$(./tomcat-check.sh)
bash: ./tomcat-check.sh: No such file or directory
ec2-user@ip-192-168-1-173 ~]$ VALUE=$(./apache-check.sh)
ec2-user@ip-192-168-1-173 ~]$ aws cloudwatch put-metric-data \
--metric-name ApacheStatus \
--namespace CustomMetrics \
--value $VALUE
ec2-user@ip-192-168-1-173 ~]$ AC
ec2-user@ip-192-168-1-173 ~]$
```


- Create a new dashboard and select custom metrics and select apache status



- In dashboard we can see if it runs we get 1 else if it fails 0



12. If CPU utilization is more than 70%, then it should trigger auto scaling and launch new instance.

- Created new Launch template

Launch Template ID	Launch Template Name	Default Version	Latest Version	Create Time	Created By
lt-01ee8c30db1e6f440	New-Template	1	1	2026-04-28T06:19:03.000Z	arn:aws:iam::928:

- Created new Auto scaling group

Name	Status	Launch template/configuration	Instances	Instance health	Desired capacity	Min	Max
My-Autoscale	At desired capacity	New-Template Version Default	2	2/2 Healthy	2	1	4

- Two new desired instances created default after giving desired -2, mini-1, max-4

Successfully initiated termination (deletion) of i-0644adde138858efe,j-059ff97583a87583a

Instances (4) Info Last updated less than a minute ago Connect Instance state Actions

Saved filter sets: Running Q X

Instance state = running X Clear filters

☐	Name 🔗	Instance ID	Instance state	Instance type	Status check	Alarm status	Availab
☐		i-0a6c0ef3892476d60	Running 🔗 🔗	t3.micro	Initializing 🕒	View alarms +	us-east-
☐		i-0025cb6c0494330c0	Running 🔗 🔗	t3.micro	Initializing 🕒	View alarms +	us-east-
☐	Public-EC2	i-04c738b3c0430274c	Running 🔗 🔗	t3.micro	3/3 checks passed ✅	View alarms +	us-east-
☐	Private-EC2	i-0c12edac4eeaacd2a	Running 🔗 🔗	t3.micro	3/3 checks passed ✅	View alarms +	us-east-

- In Auto scaling group by selecting open it and in that activity we can see

Activity history (2) 🔗

Q Filter activity history Filter by a date and time range < 1 > ⚙️

Status	Description	Cause	Start time	End time
Successful ✅	Launching a new EC2 instance: i-0a6c0ef3892476d60	At 2026-04-28T08:43:27Z a user request created an AutoScalingGroup changing the desired capacity from 0 to 2. At 2026-04-28T08:43:28Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 0 to 2.	2026 April 28, 02:13:30 PM +05:30	2026 April 28, 02:13:38 PM +05:30
Successful ✅	Launching a new EC2 instance: i-0025cb6c0494330c0	At 2026-04-28T08:43:27Z a user request created an AutoScalingGroup changing the desired capacity from 0 to 2. At 2026-04-28T08:43:28Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 0 to 2.	2026 April 28, 02:13:30 PM +05:30	2026 April 28, 02:13:40 PM +05:30

- After terminated the default ec2 and we can see new instance created in instance

Instance state = running X Clear filters

< 1

☐	Name 🔗	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	P
☐		i-0e23b9ff6238adce6	Running 🔗 🔗	t3.micro	Initializing 🕒	View alarms +	us-east-1d	-
☐	Public-EC2	i-04c738b3c0430274c	Running 🔗 🔗	t3.micro	3/3 checks passed ✅	View alarms +	us-east-1d	-
☐	Private-EC2	i-0c12edac4eeaacd2a	Running 🔗 🔗	t3.micro	3/3 checks passed ✅	View alarms +	us-east-1c	-

- In Auto scale group also in active we can see new update successful one

Activity history (4) 🔗

Q Filter activity history Filter by a date and time range < 1 > ⚙️

Status	Description	Cause	Start time	End time
Successful ✅	Launching a new EC2 instance: i-0e23b9ff6238adce6	At 2026-04-28T08:53:32Z an instance was launched in response to an unhealthy instance needing to be replaced.	2026 April 28, 02:23:34 PM +05:30	2026 April 28, 02:24:06 PM +05:30
Connection draining in progress 🔄	Terminating EC2 instance: i-0025cb6c0494330c0 - Waiting For ELB Connection Draining.	At 2026-04-28T08:53:32Z an instance was taken out of service in response to an EC2 health check indicating it has been terminated or stopped.	2026 April 28, 02:23:32 PM +05:30	
Successful ✅	Launching a new EC2 instance: i-0a6c0ef3892476d60	At 2026-04-28T08:43:27Z a user request created an AutoScalingGroup changing the desired capacity from 0 to 2. At 2026-04-28T08:43:28Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 0 to 2.	2026 April 28, 02:13:30 PM +05:30	2026 April 28, 02:13:38 PM +05:30
Successful ✅	Launching a new EC2 instance: i-0025cb6c0494330c0	At 2026-04-28T08:43:27Z a user request created an AutoScalingGroup changing the desired capacity from 0 to 2. At 2026-04-28T08:43:28Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 0 to 2.	2026 April 28, 02:13:30 PM +05:30	2026 April 28, 02:13:40 PM +05:30

- After giving more load on CPU utilization, we can see new instances created

```

18 root      20   0   0   0   0 S   0.0   0.0
[ec2-user@ip-192-168-1-173 ~]$ stress --cpu 4 --timeout 300
stress: info: [10045] dispatching hogs: 4 cpu, 0 io, 0 vm, 0
AC
[ec2-user@ip-192-168-1-173 ~]$ yes > /dev/null &
[5] 10254
[ec2-user@ip-192-168-1-173 ~]$ yes > /dev/null &
[6] 10255
[ec2-user@ip-192-168-1-173 ~]$ yes > /dev/null &
[7] 10256
[ec2-user@ip-192-168-1-173 ~]$ yes > /dev/null &
[8] 10257
[ec2-user@ip-192-168-1-173 ~]$ yes > /dev/null &
[9] 10258
[ec2-user@ip-192-168-1-173 ~]$

```

- We can see in Auto Scaling Group

Activity history (5)		
Filter activity history		Filter by a date and time range
Status	Description	Cause
✓ Successful	Terminating EC2 instance: i-0a6c0ef3892476d60	At 2026-04-28T08:55:30Z an instance was taken out of service in response to an EC2 health check indicating it has been terminated or stopped.
✓ Successful	Launching a new EC2 instance: i-0e23b9ff6238adce6	At 2026-04-28T08:53:32Z an instance was launched in response to an unhealthy instance needing 1 replaced.
✓ Successful	Terminating EC2 instance: i-0025cb6c0494330c0	At 2026-04-28T08:53:32Z an instance was taken out of service in response to an EC2 health check indicating it has been terminated or stopped.
✓ Successful	Launching a new EC2 instance: i-0a6c0ef3892476d60	At 2026-04-28T08:43:27Z a user request created an AutoScalingGroup changing the desired capacity from 0 to 2. At 2026-04-28T08:43:28Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 0 to 2.
✓ Successful	Launching a new EC2 instance: i-0025cb6c0494330c0	At 2026-04-28T08:43:27Z a user request created an AutoScalingGroup changing the desired capacity from 0 to 2. At 2026-04-28T08:43:28Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 0 to 2.